PORTAFOLIO TÉCNICO

Pruebas unitarias en C#

API de incidencias de un festival

Objetivo	Desarrollar y ejecutar pruebas unitarias sobre la lógica de una API REST para gestionar incidencias de un festival cultural.
Tecnologías	C#, .NET 10, xUnit y repositorio en memoria para preparar los escenarios de prueba.
Alcance	11 pruebas unitarias que cubren consultas, creación, validaciones, errores de negocio, asignación, resolución y resumen del festival.
Enfoque	Comprobar comportamientos esperados y situaciones en las que el servicio debe rechazar una operación.

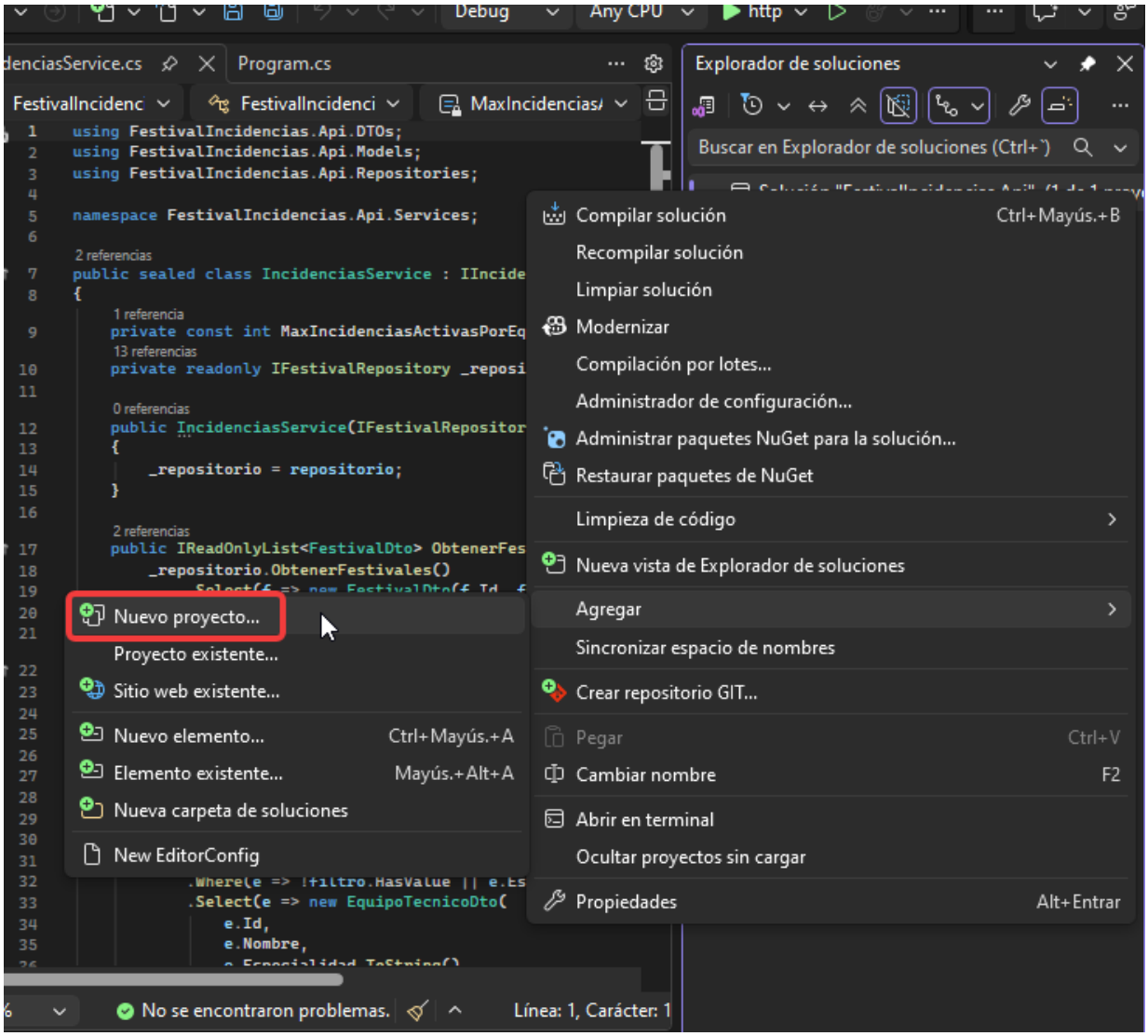
La documentación recoge el proceso de configuración, ejemplos de pruebas y evidencias de ejecución.

Armando Merino

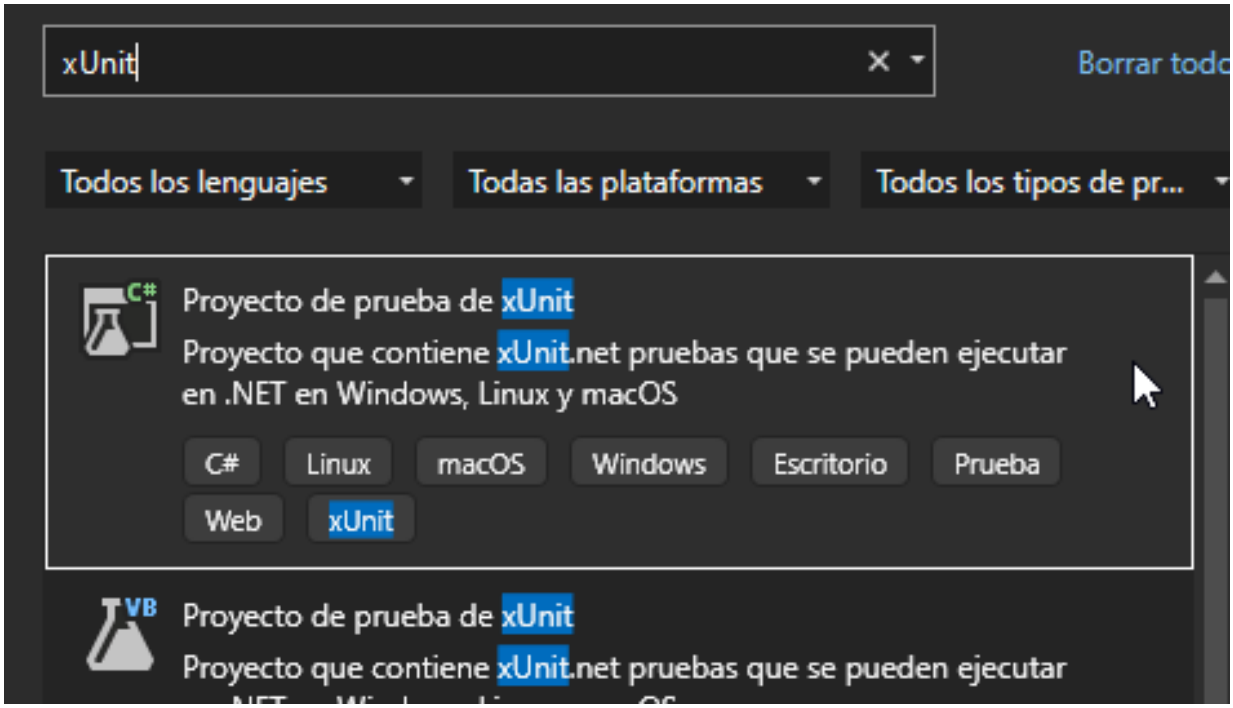
Configuración del proyecto de pruebas

Paso 1: Crear el proyecto de pruebas.

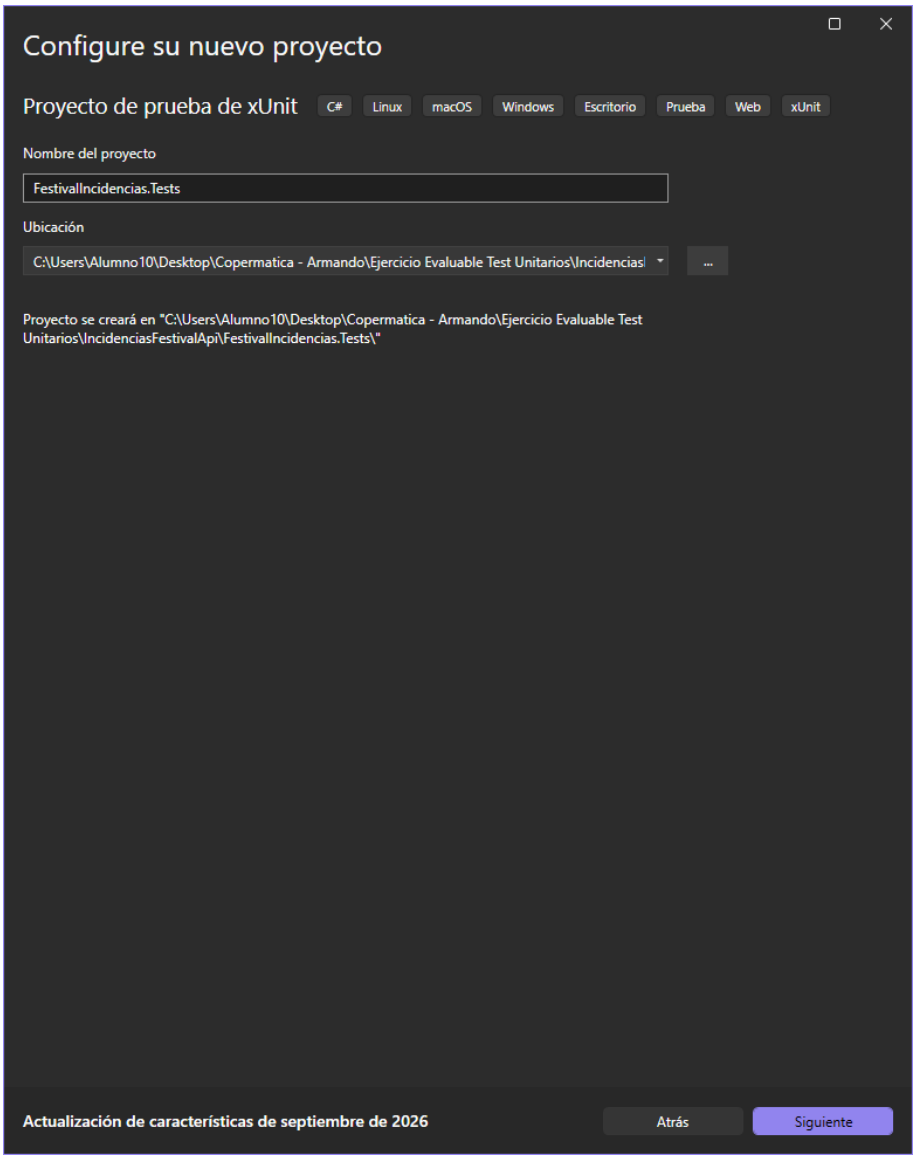
En el explorador de soluciones agregamos un nuevo proyecto test para las pruebas unitarias.



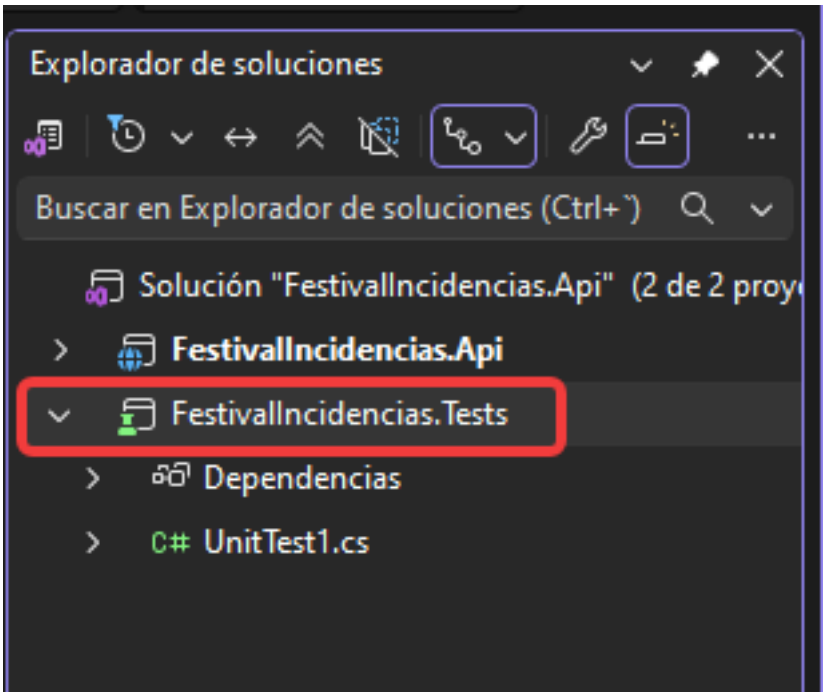
Evidencia del proyecto o de la ejecución de las pruebas.



Evidencia del proyecto o de la ejecución de las pruebas.

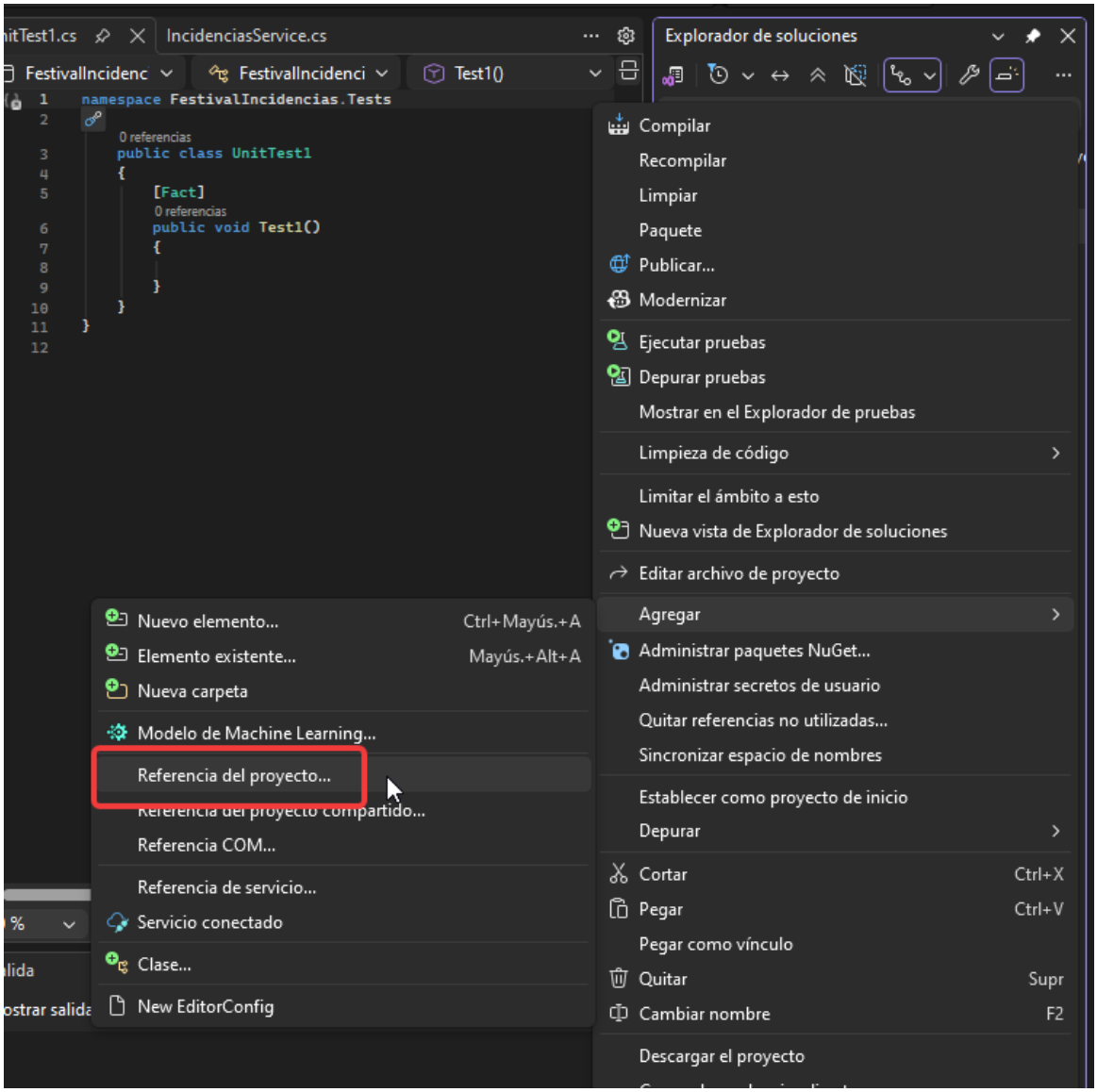


Evidencia del proyecto o de la ejecución de las pruebas.

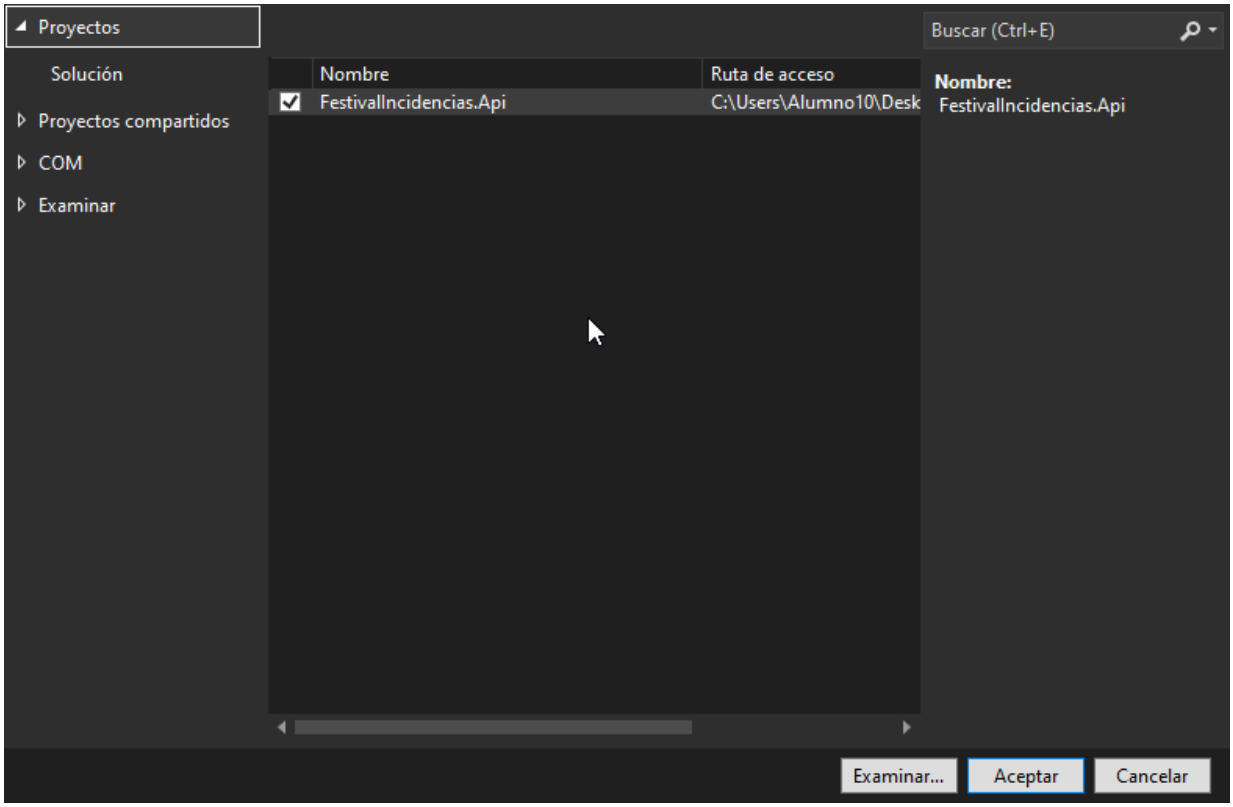


Evidencia del proyecto o de la ejecución de las pruebas.

Ahora añadimos la referencia al proyecto original para poder probar sus clases.



Evidencia del proyecto o de la ejecución de las pruebas.



Evidencia del proyecto o de la ejecución de las pruebas.

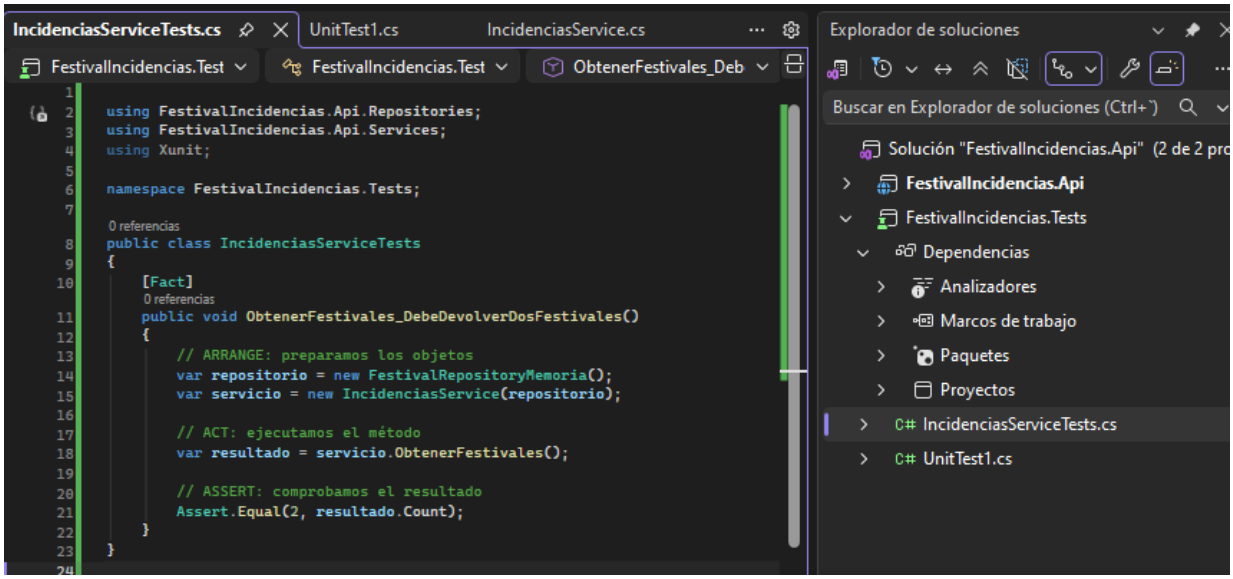
Primeras pruebas: consulta de festivales e incidencias

Una vez creado el proyecto FestivalIncidencias.Test

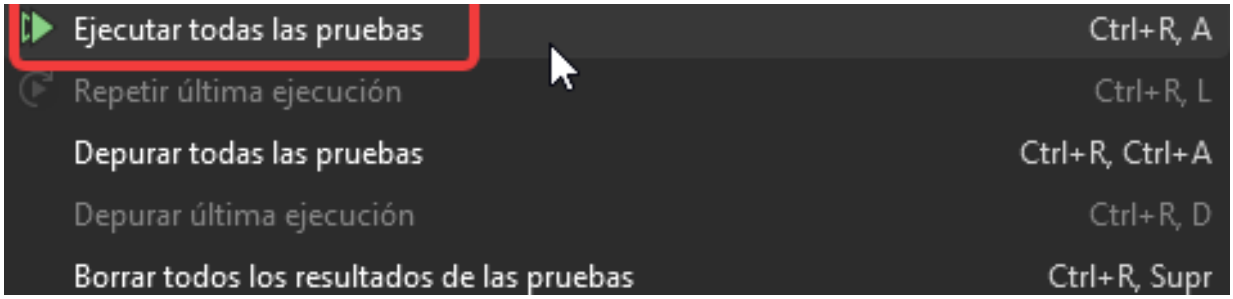
He creado un archivo IncidenciasService.Test.cs y contiene dos festivales: uno activo y otro inactivo.

Ahora ejecutamos todas las pruebas para ver si las pruebas unitarias han pasado la prueba:

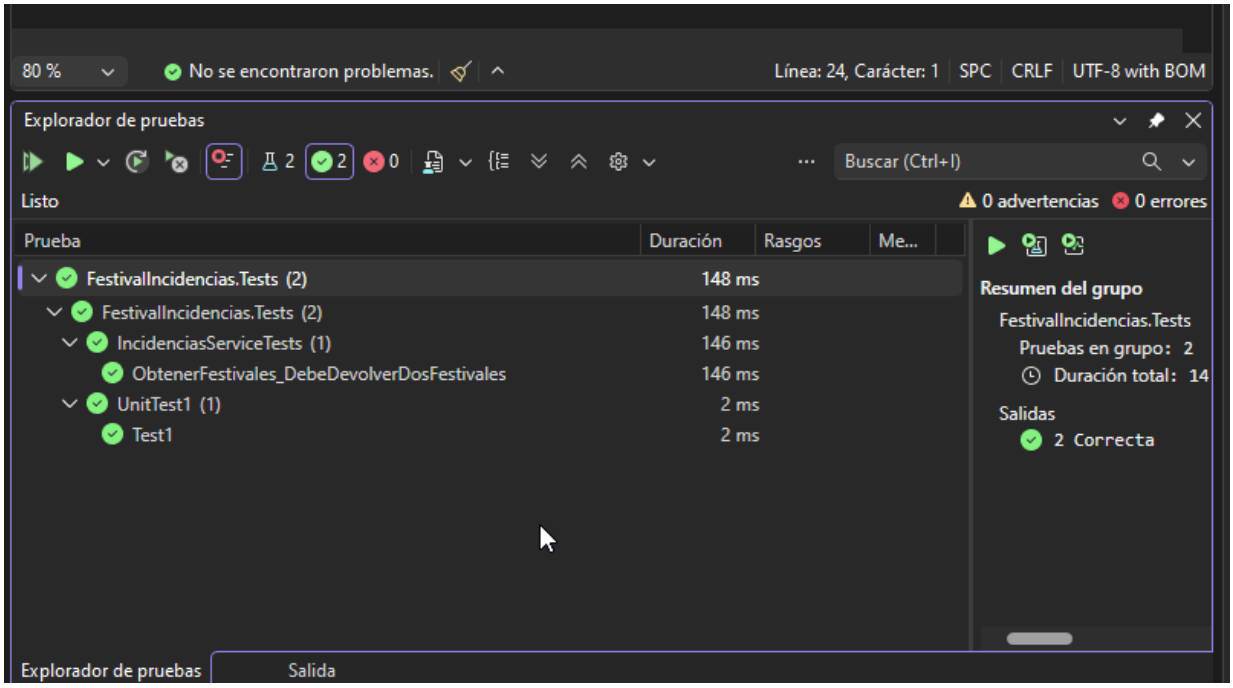
Han pasado la prueba, eso significa que nuestro proyecto test esta bien conectado con la API, xUnit esta funcionando y el servicio devuelve los dos festivales que esperábamos.



Evidencia del proyecto o de la ejecución de las pruebas.



Evidencia del proyecto o de la ejecución de las pruebas.



Evidencia del proyecto o de la ejecución de las pruebas.

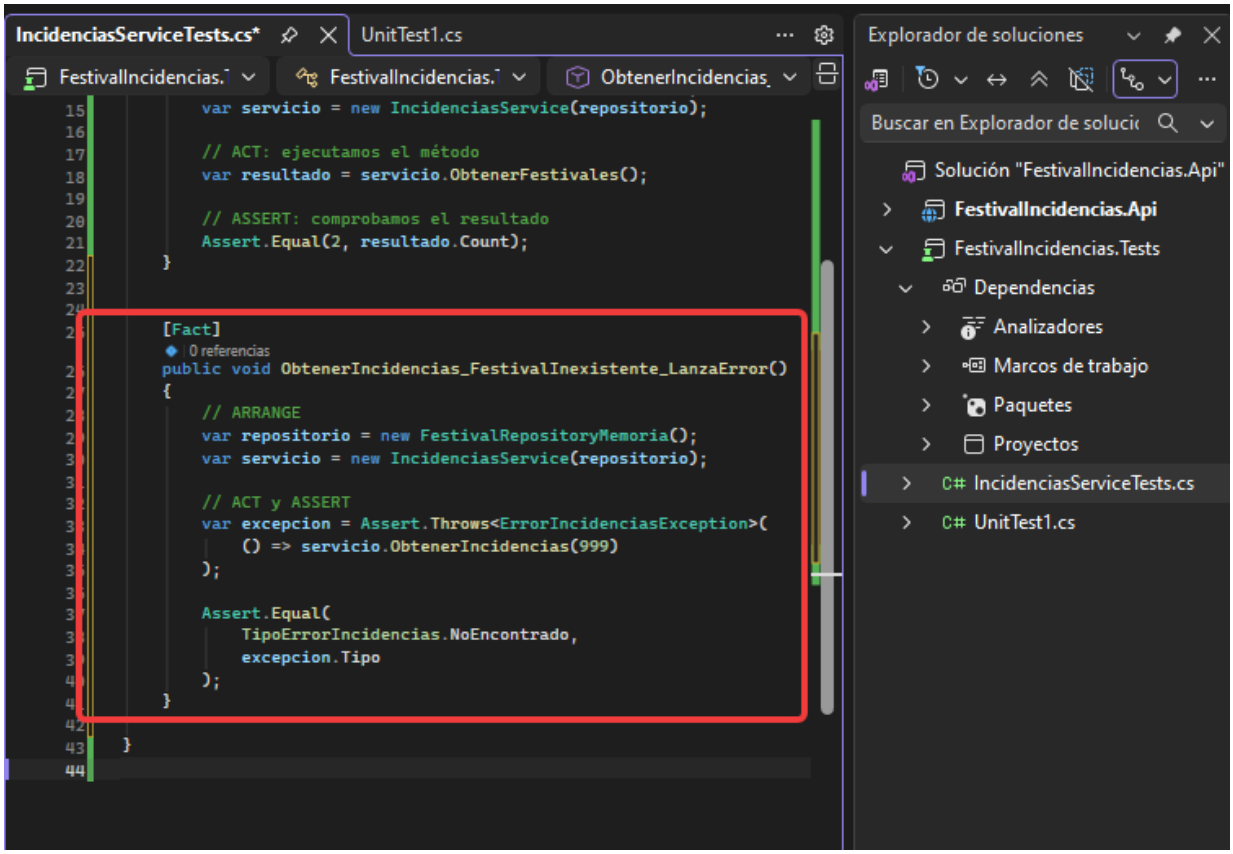
Ahora vamos a comprobar un error, es decir obtener las incidencias de un festival que no existe, debería de lanzar un error.

He añadido este código en la clase Incidencias para la comprobación del error.

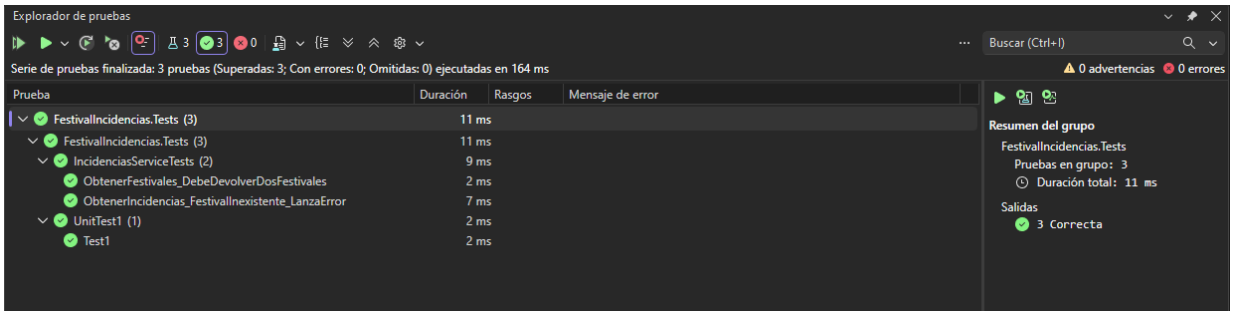
Lo que haces es consultar las incidencias de un festival con id 999 que claramente no existe.

El resultado esperado era que el servicio lanzase un error y lanzo la excepción experada.

Ha pasado las pruebas:



Evidencia del proyecto o de la ejecución de las pruebas.



Evidencia del proyecto o de la ejecución de las pruebas.

Creación de incidencias y validaciones

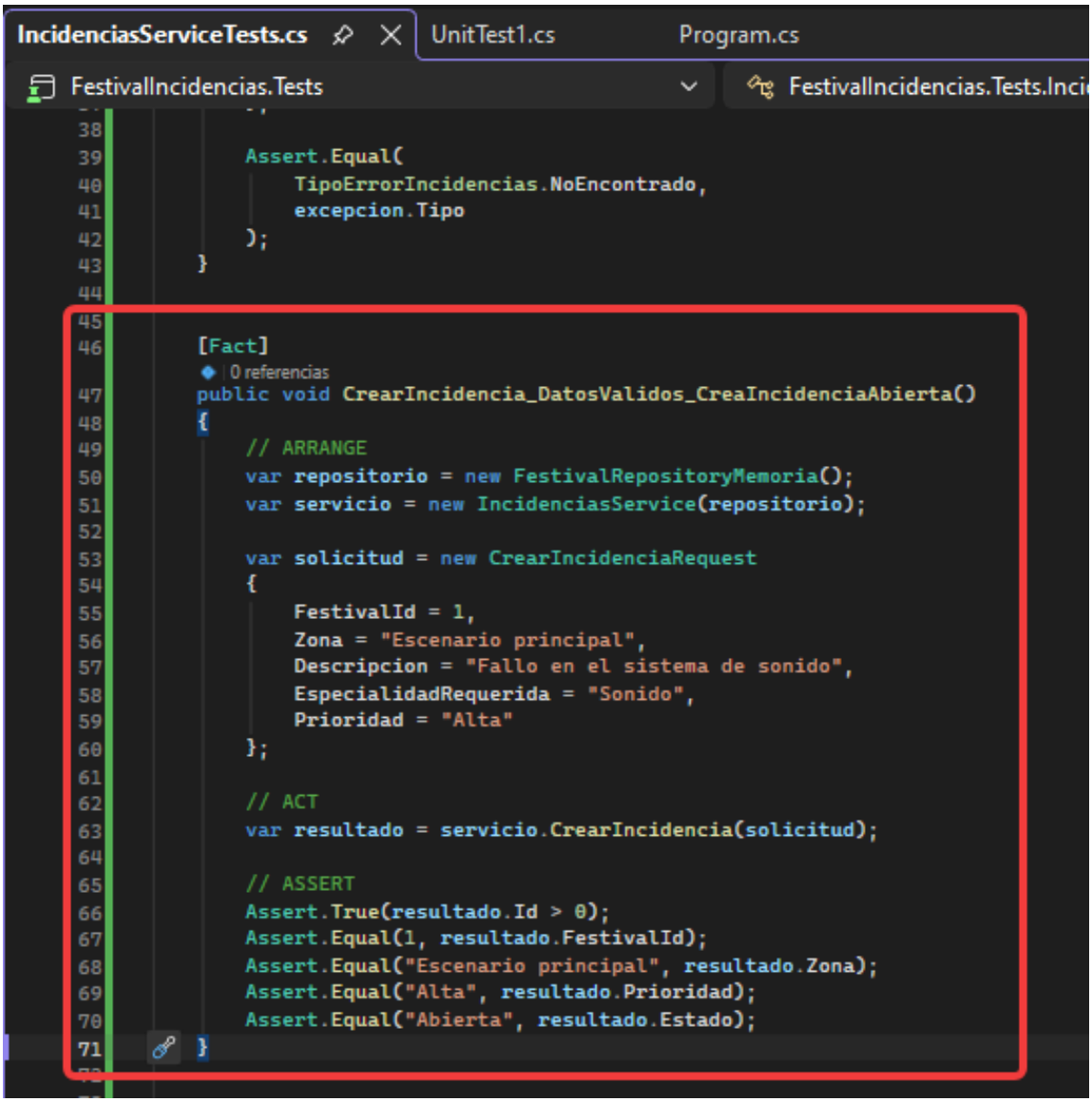
Ahora vamos a probar la creación de incidencias.

Vamos a probar un caso valido y otro que debe producir un error.

En este caso vamos a probar primero la creación correcta de una incidencia y después el rechazo de una incidencia para un festival inactivo.

Ahora bien, en la clase FestivalIncidencias he añadido este código:

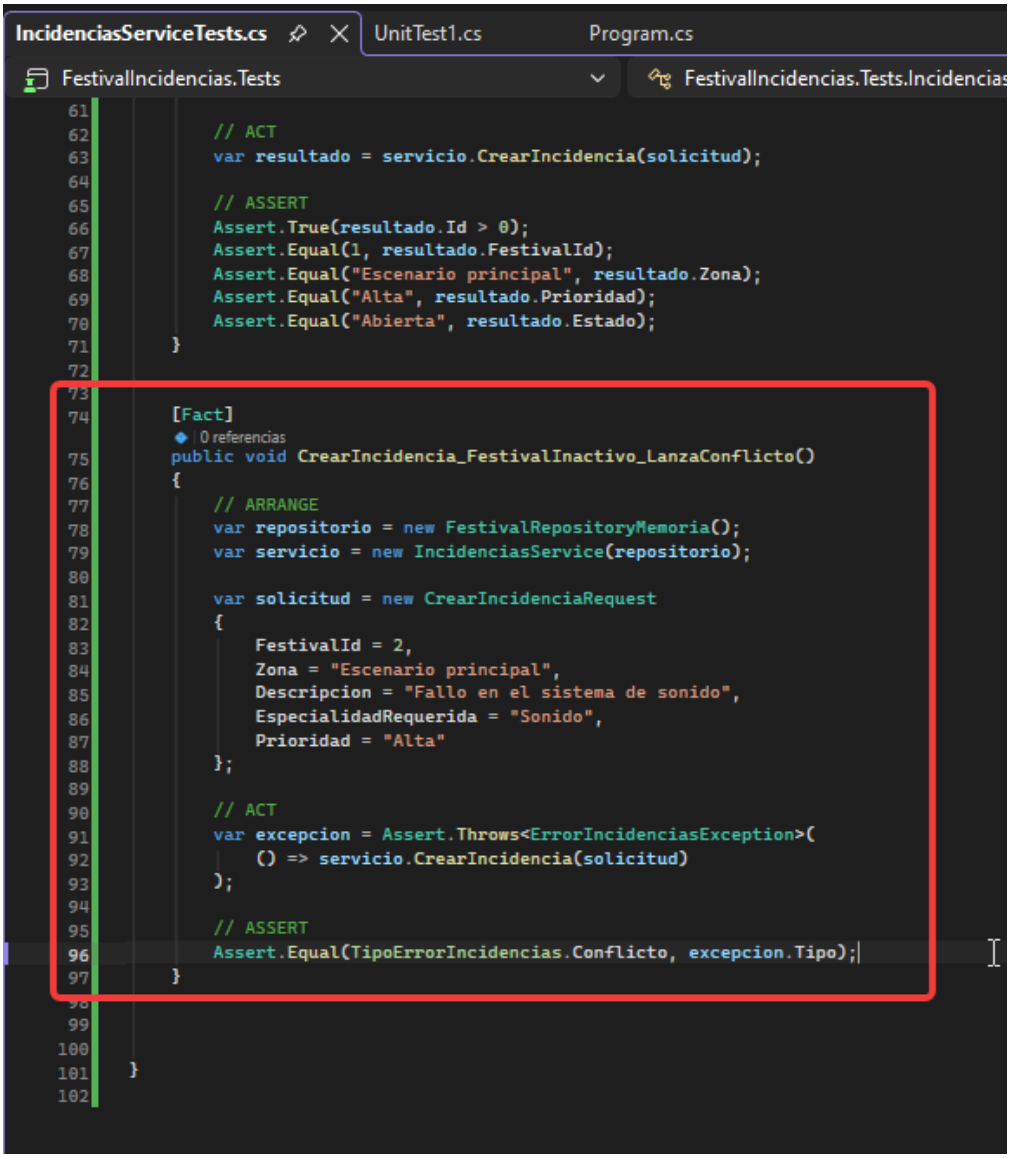
Comprueba que cuando los datos son correctos y el festival esta activo, la incidencia se crea, recibe el id y el comienza con el estado abierta.



Evidencia del proyecto o de la ejecución de las pruebas.

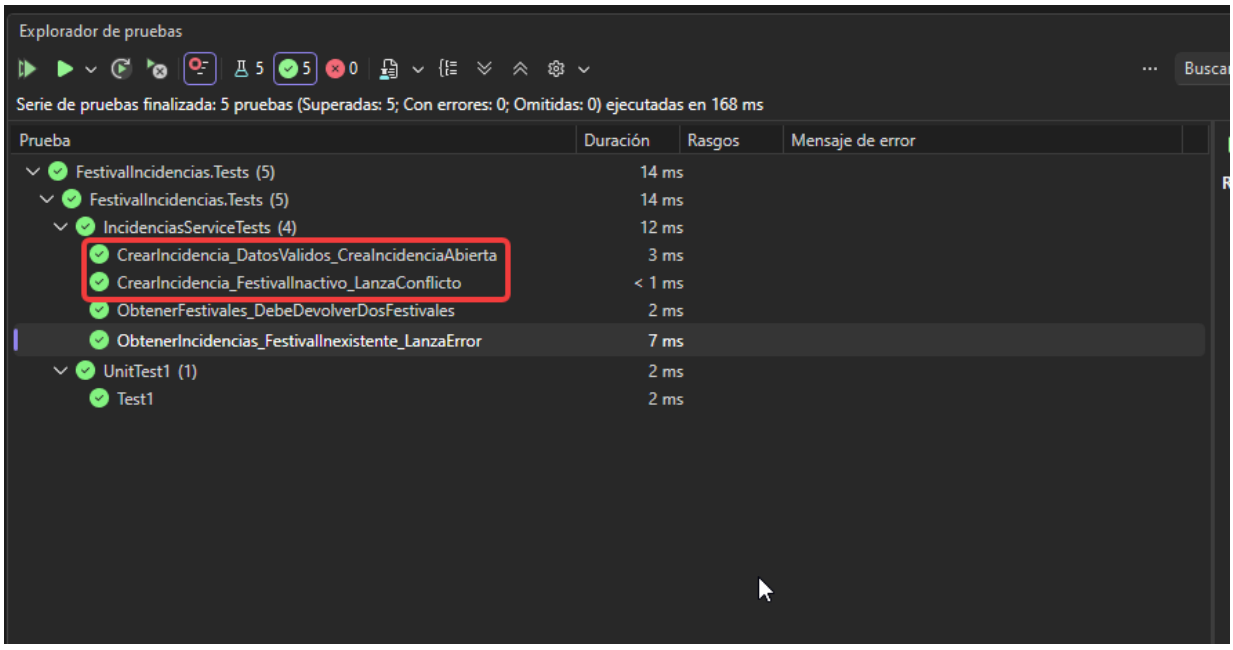
La otra incidencia que he creado es esta:

Rechazar una incidencia en un festival inactivo,



Evidencia del proyecto o de la ejecución de las pruebas.

Una vez ejecutadas todas las pruebas unitarias, han pasado la prueba con éxito.



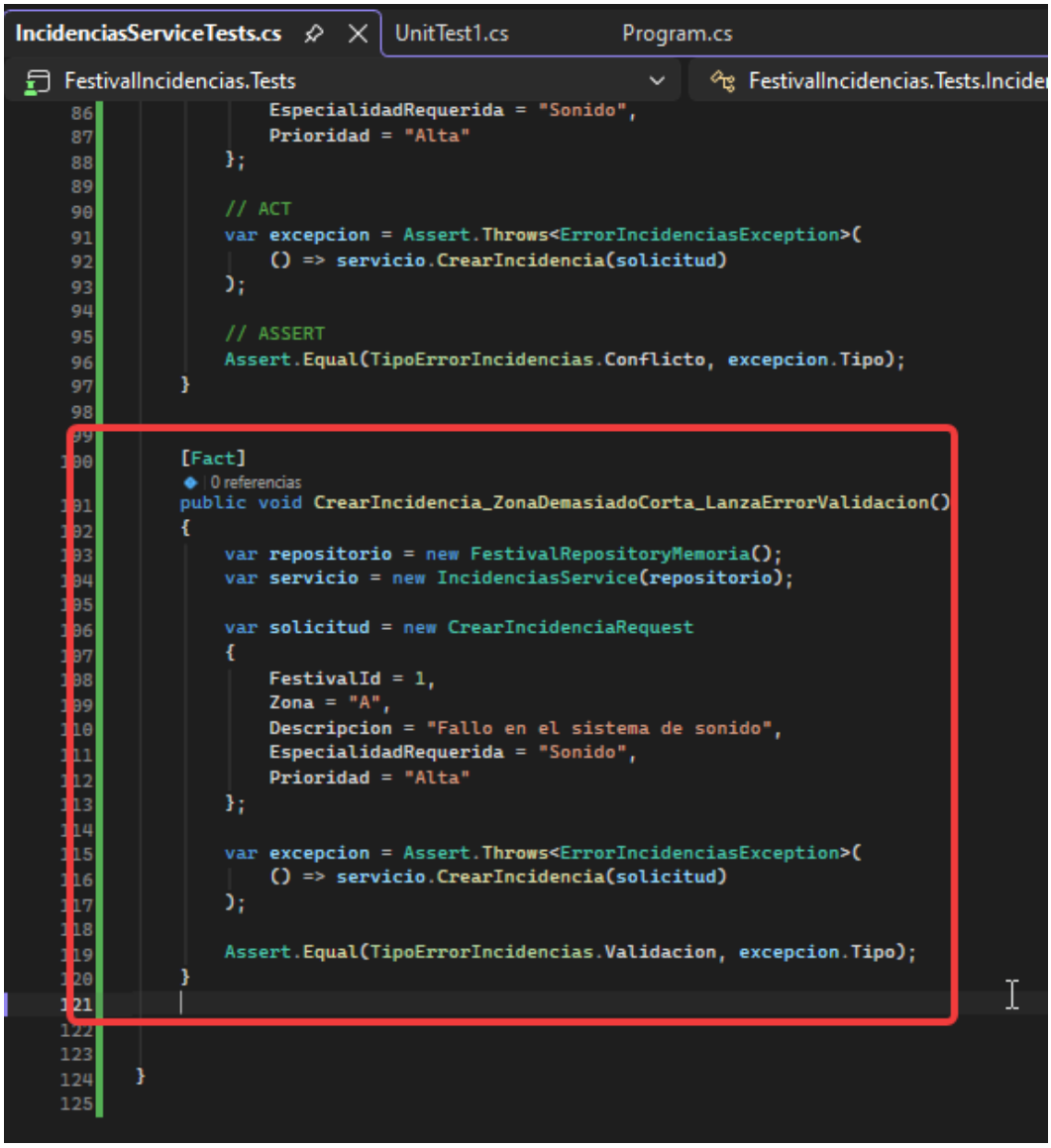
Evidencia del proyecto o de la ejecución de las pruebas.

De momento llevamos 4 pruebas unitarias que funcionan.

Ahora vamos a crear una incidencia con una zona demasiado corta.

En la clase IncidenciasService, la zona pone que debe tener entre 2 y 80 caracteres.

Entonces vamos a comprobar que si tiene solo un carácter el servicio lanza un error de validación.



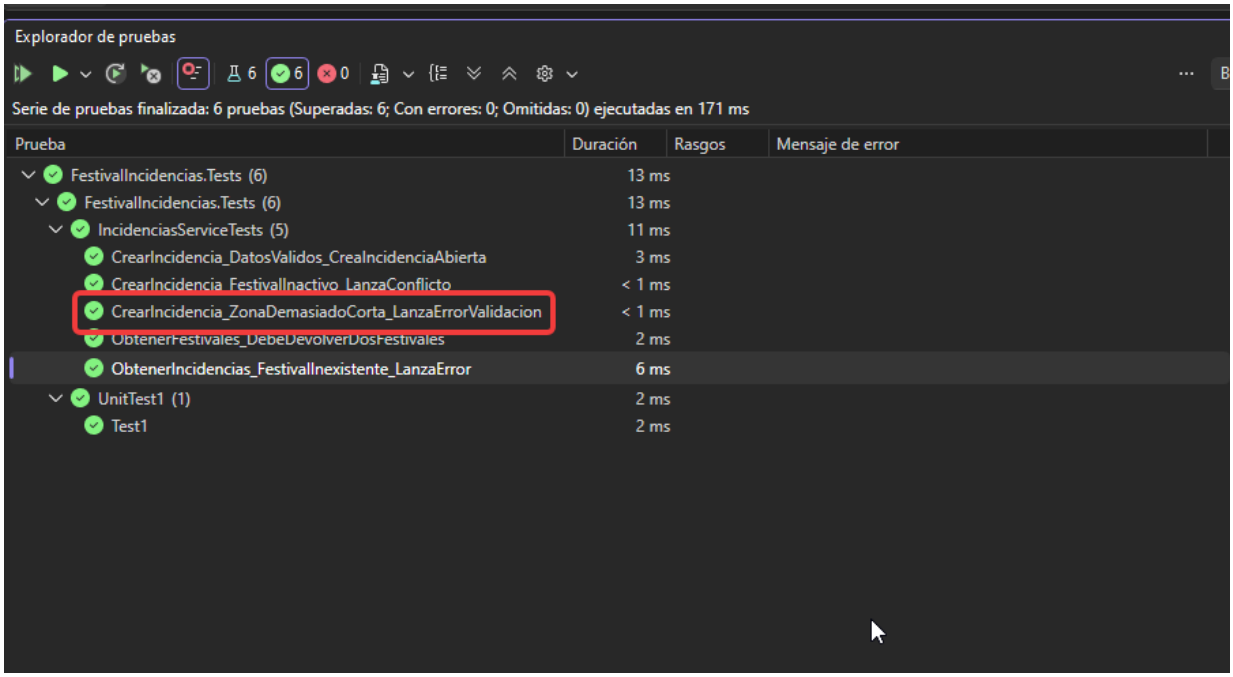
Evidencia del proyecto o de la ejecución de las pruebas.

Ha pasado el test.

Vamos a añadir otra prueba más, en este caso una descripción demasiado corta.

El servicio exige que la descripción tenga entre 10 y 500 caracteres. Pues vamos a comprobar que una descripción que tenga pocos caracteres y de un error de validación.

He añadido este método en la clase IncidenciasServiceTests.cs



Evidencia del proyecto o de la ejecución de las pruebas.

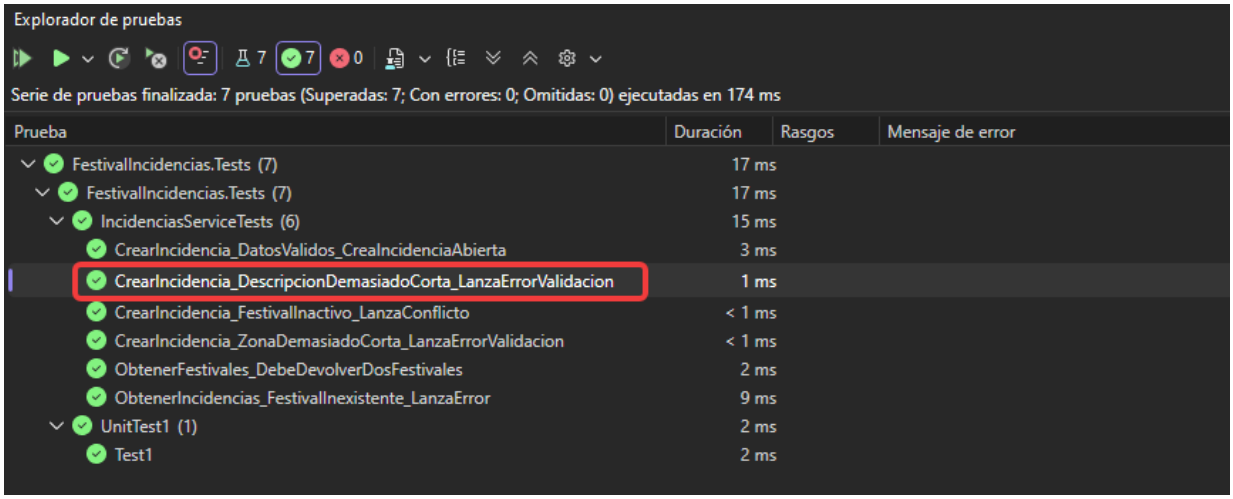
```
[Fact]
0 referencias
public void CrearIncidencia_DescripcionDemasiadoCorta_LanzaErrorValidacion()
{
    var repositorio = new FestivalRepositoryMemoria();
    var servicio = new IncidenciasService(repositorio);

    var solicitud = new CrearIncidenciaRequest
    {
        FestivalId = 1,
        Zona = "Escenario principal",
        Descripcion = "Fallo",
        EspecialidadRequerida = "Sonido",
        Prioridad = "Alta"
    };

    var excepcion = Assert.Throws<ErrorIncidenciasException>(
        () => servicio.CrearIncidencia(solicitud)
    );

    Assert.Equal(TipoErrorIncidencias.Validacion, excepcion.Tipo);
}
```

Evidencia del proyecto o de la ejecución de las pruebas.



Evidencia del proyecto o de la ejecución de las pruebas.

Validaciones adicionales, asignación y resolución

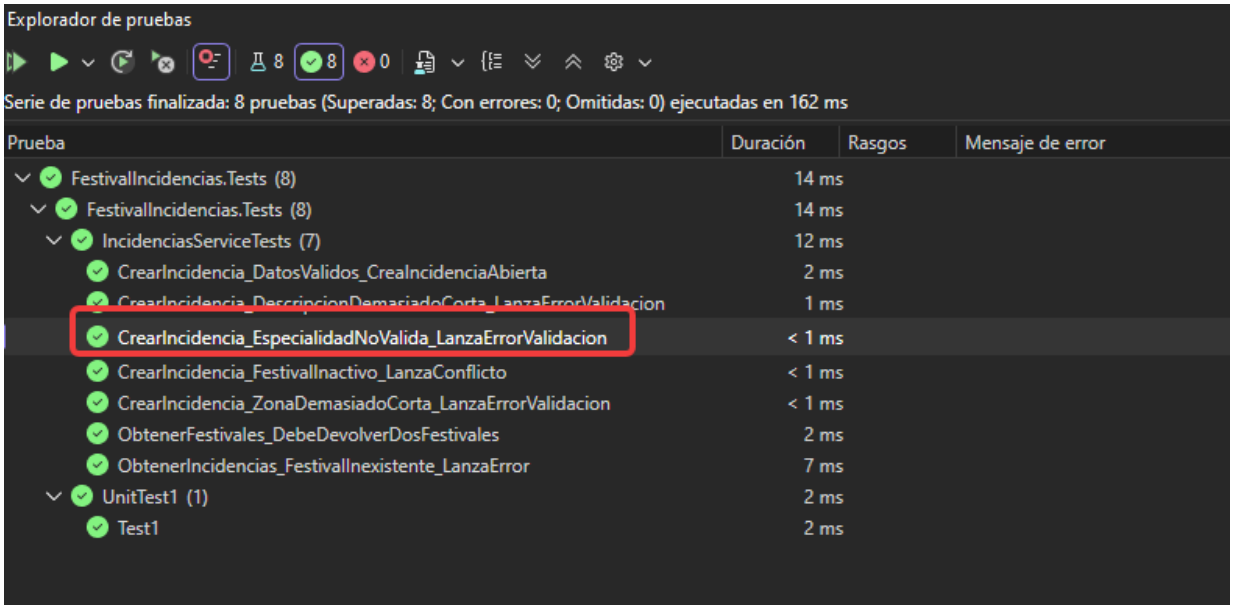
Ahora vamos a añadir una 7 validación, en este caso vamos a validar que la especialidad indicada existe en el enum EspecialidadTecnica y voy a comprobar que rechaza una especialidad inventada.

He añadido el siguiente método en la misma clase en IncidenciasServiceTests.cs

Y ha pasado los test:

La siguiente validación será que no se pueda asignar una incidencia a un equipo técnico que no existe.

Intentamos asignar la incidencia con id 999 y eso no existe al equipo con id 1 y el servicio debería de comprobar primero si existe la incidencia entonces debe de dar una excepción de tipo NoEncontrado



Evidencia del proyecto o de la ejecución de las pruebas.

```
[Fact]
0 referencias
public void AsignarIncidencia_IncidenciaInexistente_LanzaError()
{
    var repositorio = new FestivalRepositoryMemoria();
    var servicio = new IncidenciasService(repositorio);

    var solicitud = new AsignarIncidenciaRequest
    {
        EquipoId = 1
    };

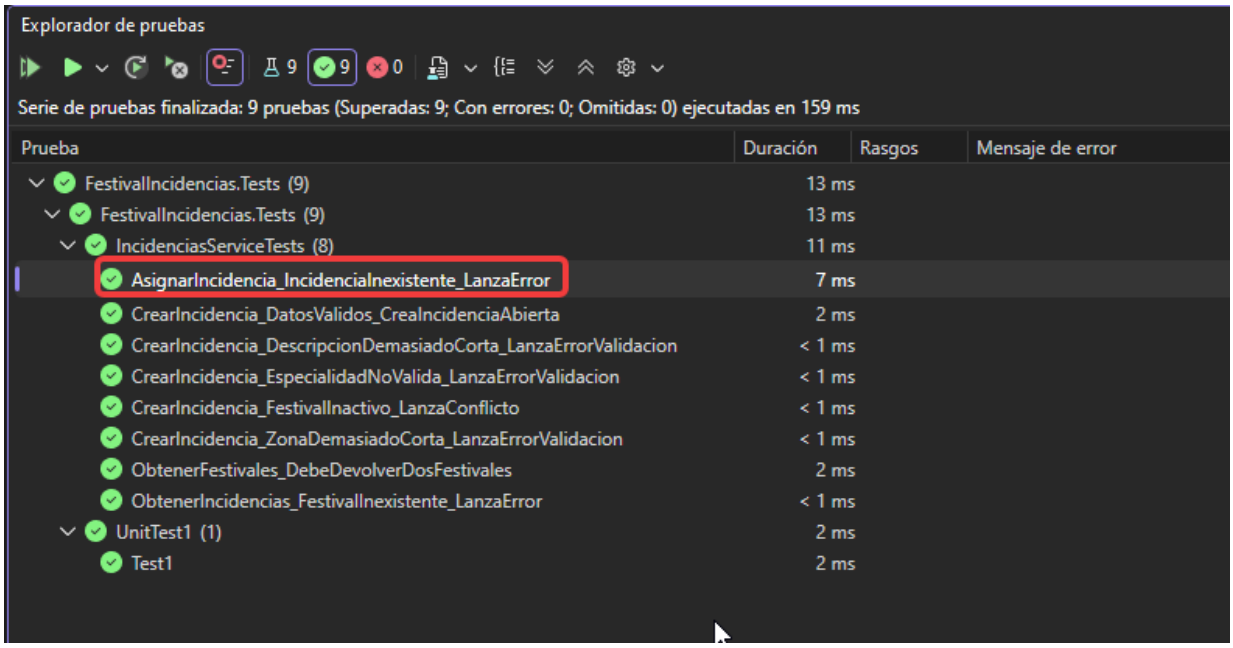
    var excepcion = Assert.Throws<ErrorIncidenciasException>(
        () => servicio.AsignarIncidencia(999, solicitud)
    );

    Assert.Equal(TipoErrorIncidencias.NoEncontrado, excepcion.Tipo);
}
```

Evidencia del proyecto o de la ejecución de las pruebas.

Y ha pasado las pruebas unitarias:

La siguiente prueba unitaria intentara resolver una incidencia que no está asignada y debe de dar un error ya que tiene que tener asignado un equipo técnico.



Evidencia del proyecto o de la ejecución de las pruebas.



```
[Fact]
0 referencias
public void ResolverIncidencia_NoAsignada_LanzaConflicto()
{
    var repositorio = new FestivalRepositoryMemoria();
    var servicio = new IncidenciasService(repositorio);

    var solicitudCreacion = new CrearIncidenciaRequest
    {
        FestivalId = 1,
        Zona = "Escenario principal",
        Descripcion = "Fallo en el sistema de sonido",
        EspecialidadRequerida = "Sonido",
        Prioridad = "Alta"
    };

    var incidencia = servicio.CrearIncidencia(solicitudCreacion);

    var solicitudResolucion = new ResolverIncidenciaRequest
    {
        Resolucion = "Se ha revisado el equipo"
    };

    var excepcion = Assert.Throws<ErrorIncidenciasException>(
        () => servicio.ResolverIncidencia(
            incidencia.Id, solicitudResolucion)
    );

    Assert.Equal(TipoErrorIncidencias.Conflicto, excepcion.Tipo);
}
```

Evidencia del proyecto o de la ejecución de las pruebas.

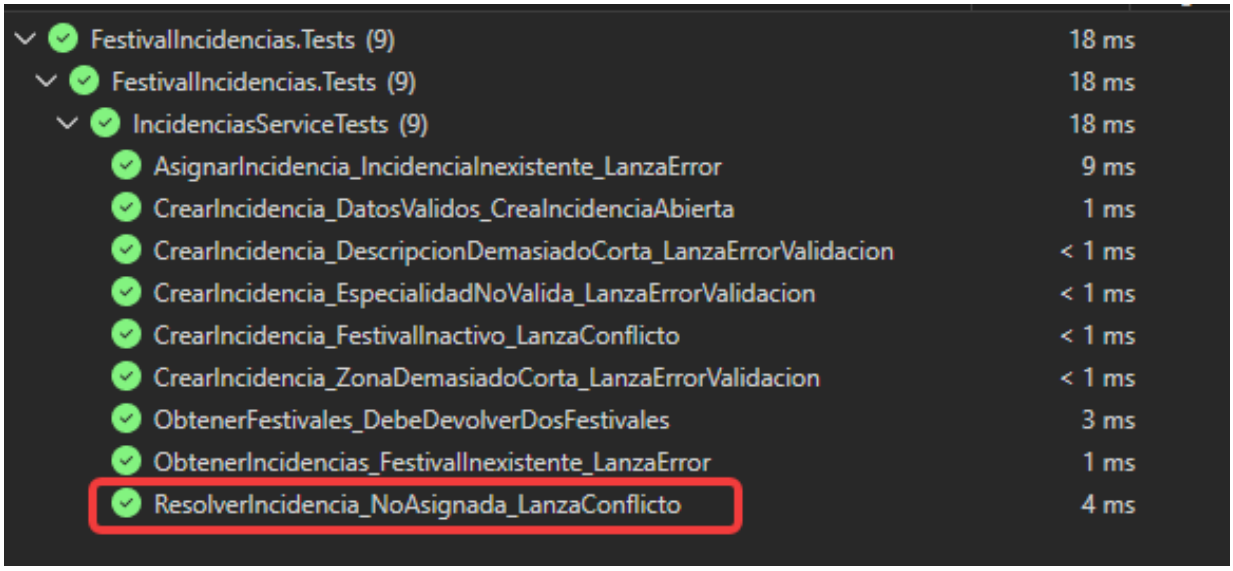
Ha pasado las pruebas.

La siguiente prueba unitaria se encargará de comprobar que el servicio rechaza una resolución demasiado corta.

Esta vez probaremos otra validación del método ResolverIncidencia, la resolución debe tener entre 10 y 500 caracteres. Pero primero para llegar a esa validación tendremos que crear una incidencia y asignarla a un equipo.

Aquí creamos una incidencia valida, buscamos un equipo con la especialidad sonido que tenga menos de 3 incidencias activas y asignamos la incidencia ese equipo.

Intentamos resolverla con “Fallo”, que tiene menos de 10 caracteres y finalmente comprobamos que el servicio lanza un error de tipo validación.



Evidencia del proyecto o de la ejecución de las pruebas.

```
[Fact]
0 referencias
public void ResolverIncidencia_ResolucionDemasiadoCorta_LanzaErrorValidacion()
{
    var repositorio = new FestivalRepositoryMemoria();
    var servicio = new IncidenciasService(repositorio);

    var solicitudCreacion = new CrearIncidenciaRequest
    {
        FestivalId = 1,
        Zona = "Escenario principal",
        Descripcion = "Fallo en el sistema de sonido",
        EspecialidadRequerida = "Sonido",
        Prioridad = "Alta"
    };

    var incidencia = servicio.CrearIncidencia(solicitudCreacion);

    var equipo = servicio.ObtenerEquipos("Sonido")
        .First(e => e.Activo && e.IncidenciasActivas < 3);

    servicio.AsignarIncidencia(
        incidencia.Id,
        new AsignarIncidenciaRequest
        {
            EquipoId = equipo.Id
        }
    );

    var solicitudResolucion = new ResolverIncidenciaRequest
    {
        Resolucion = "Fallo"
    };

    var excepcion = Assert.Throws<ErrorIncidenciasException>(
        () => servicio.ResolverIncidencia(
            incidencia.Id, solicitudResolucion)
    );

    Assert.Equal(TipoErrorIncidencias.Validacion, excepcion.Tipo);
}
```

Tabulación to

Evidencia del proyecto o de la ejecución de las pruebas.

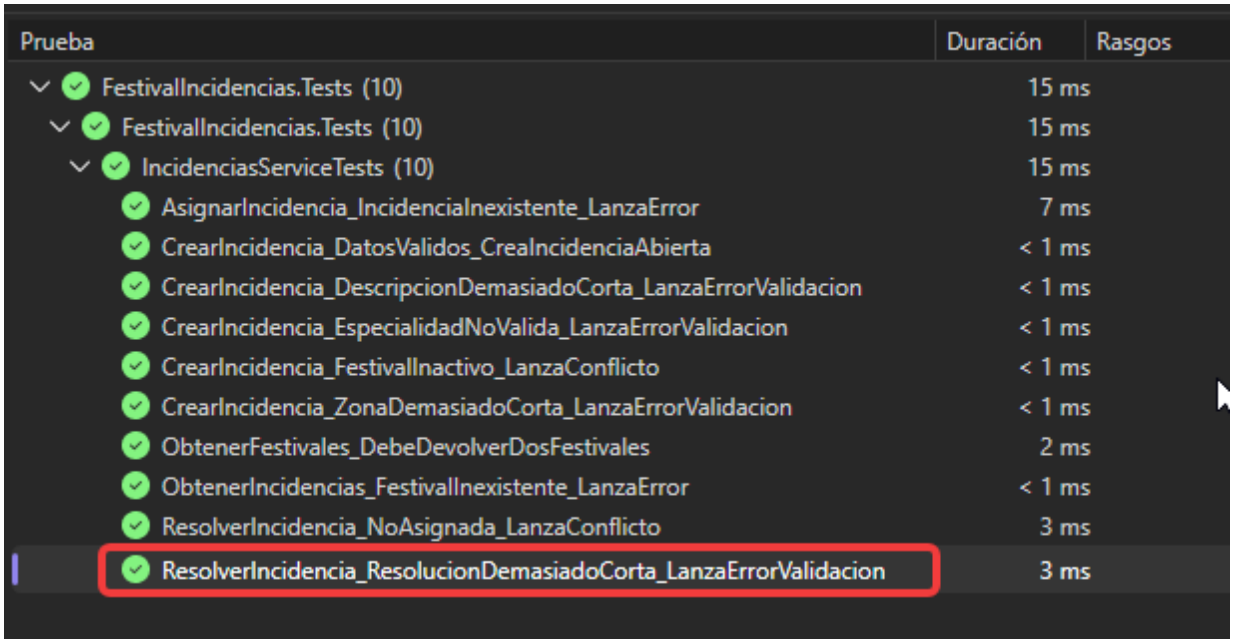
Resumen del festival y cierre

Ha pasado las pruebas:

Hemos probado casos correctos y casos de error de la API, entonces ahora voy a probar un resumen del festival, por que nos permite comprobar que el servicio cuenta correctamente las incidencias según su estado.

Comprueba que se puede consultar el resumen del festival con ID 1.

Que el resumen contiene el nombre del festival y que el total de incidencias coincide con la suma de las abiertas, asignadas y resueltas.



Evidencia del proyecto o de la ejecución de las pruebas.

```
[Fact]
0 referencias
public void ObtenerResumenFestival_FestivalExistente_DevuelveResumenCorrecto()
{
    var repositorio = new FestivalRepositoryMemoria();
    var servicio = new IncidenciasService(repositorio);

    var resultado = servicio.ObtenerResumenFestival(1);

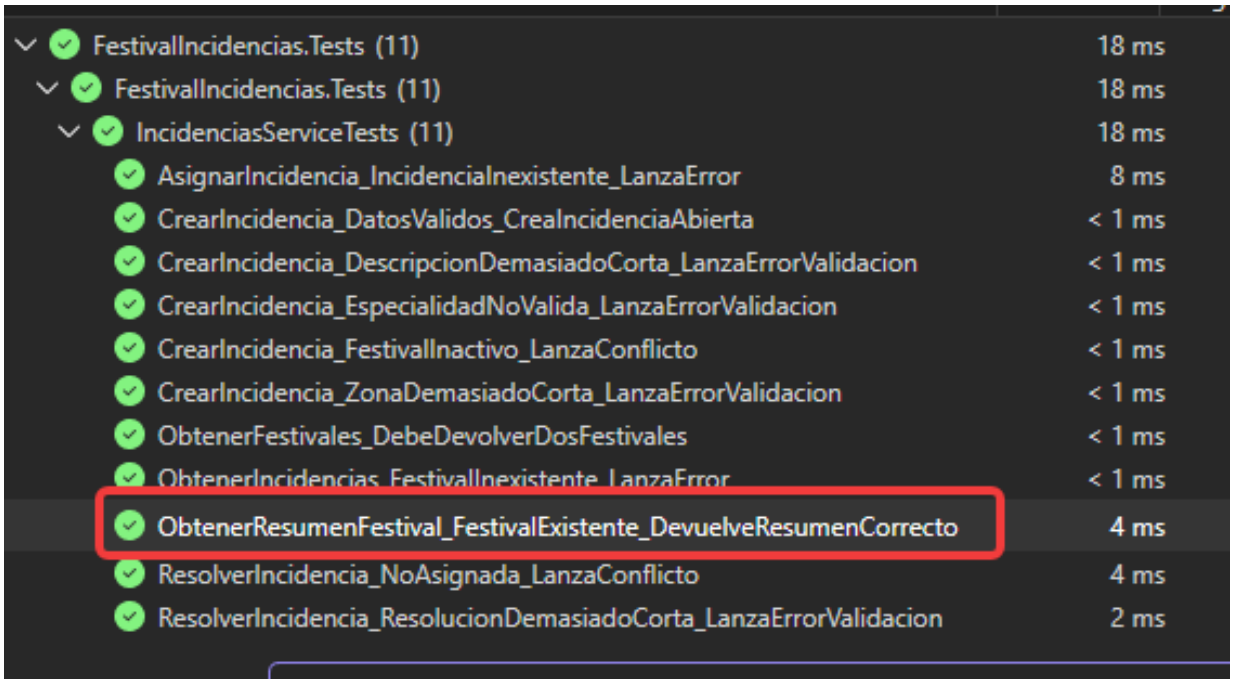
    Assert.Equal(1, resultado.FestivalId);
    Assert.False(string.IsNullOrWhiteSpace(resultado.Festival));
    Assert.Equal(
        resultado.TotalIncidencias,
        resultado.Abiertas + resultado.Asignadas + resultado.Resueltas
    );
}
```

Evidencia del proyecto o de la ejecución de las pruebas.

Ha pasado las validaciones:

Con todas estas pruebas unitarias hemos cubierto consultas, creación de incidencias,

validaciones, errores de negocio, asignación y resumen de un festival.



Evidencia del proyecto o de la ejecución de las pruebas.